APAAR ReconX

Project Understanding & Prototype Implementation Guide

SIH Problem Statement: SIH26263

Development of an APAAR ID-Based Student Data Management and Verification Platform for AICTE-Approved Institutions

A shared reference for teammates, developers, and AI coding agents

Purpose
Explain the problem, proposed solution, prototype scope, implementation workflow, requirements, expected outcomes, testing strategy, and boundaries before coding begins.

Project status: Proposed prototype design. Requirements and integration details that have not been confirmed by the problem owner or official documentation are explicitly treated as provisional.

Version 1.0 · Prepared 24 October 2026

1. Executive Summary

APAAR ReconX is a proposed secure student-data reconciliation and verification workbench for AICTE-approved institutions. Institutions would submit student-level records through a controlled workflow. The platform would validate data quality, identify discrepancies, compare enrollment figures with configured authoritative references, create review cases, support corrections, and retain an auditable history of decisions.

The project is not simply a student database. Its purpose is to help authorized users identify, investigate, resolve, and report data discrepancies in a controlled and traceable way.

The prototype should use synthetic student records and clearly labelled demonstration data. It must not imply that APAAR, AICTE, DigiLocker/NAD, or another government service has been connected unless that integration has been formally authorized and technically verified.

2. The Problem We Are Solving

Student enrollment information can arrive from different institutions and systems in different formats. Missing or malformed identifiers, repeated records, inconsistent academic-year or programme information, unexplained enrollment-count differences, and limited correction history can make verification and reporting difficult.

Example (fictional data): an institution has a configured approved intake of 120, but a submission contains 126 records. The platform should flag the six-record discrepancy, check the applicable counting rules and exceptions, and create a case for authorized review. It must not automatically conclude that the institution has violated a rule.

Key problem areas

- Missing, malformed, or repeated APAAR IDs.
- Duplicate records within a single upload.
- Possible cross-institution matches requiring careful human review.
- Missing fields and inconsistent programme, institution, or academic-year details.
- Enrollment totals that do not reconcile with the correct authoritative reference and applicable exceptions.
- Insufficient visibility into who corrected a record, who reviewed it, when the decision was made, and why.
- Difficulty producing consistent reports from verified, pending, rejected, or unresolved records.

3. Proposed Solution

APAAR ReconX provides a controlled workflow from data submission to review and reporting. It combines deterministic validation rules, discrepancy detection, human review, correction tracking, and auditability.

Core capabilities

- **Data intake:** authorized institution users upload a CSV or Excel file using a documented template.
- **Validation:** check required fields, data types, formatting, duplicates, and internal consistency.
- **Reconciliation:** compare submission totals against the correct configured reference data and documented rules.
- **Issue management:** create actionable issues with row references, reasons, severity, and status.
- **Human review:** authorized reviewers inspect evidence, request clarification, and record a decision with a reason.
- **Correction history:** retain prior and corrected values where permitted, along with user, time, and change details.
- **Reporting:** show submission counts, issue categories, unresolved cases, and review outcomes.
- **Security:** enforce role-based access, institution-level data separation, audit logging, and privacy-conscious data handling.

4. What the System Should and Should Not Claim

Capability	Correct project behavior
APAAR ID format check	Validate against a verified specification when available. Format validity alone is not identity proof.
Duplicates	Detect exact duplicates within a file or authorized dataset; label cross-institution matches as possible matches for review.
Intake reconciliation	Compare with the authoritative intake reference and applicable category/exception rules; do not assume a simple raw count is sufficient.
Student verification	Keep identifier checks, institution-attested records, current enrollment verification, and academic-award verification distinct.
Official integration	Integrate only after permission, onboarding, documentation, credentials, and access scope are confirmed. Until then, use a clearly labelled mock adapter.
Final decision	Require an authorized reviewer and record the outcome, evidence, timestamp, and reason.

Critical distinction: an APAAR ID is an identifier. It does not, by itself, prove current enrollment or establish that every submitted field is correct.

5. Prototype Workflow: End to End

1. Sign in (demo)

Use test accounts with predefined roles. Production identity management is out of scope until requirements are confirmed.

2. Select institution/programme

Use seeded synthetic institutions, programmes, academic years, and intake references.

3. Upload a file

Upload the documented CSV/XLSX template; show file name, row count, and submission status.

4. Validate the file

Check required fields, types, malformed identifiers, missing values, duplicate rows, and consistent references.

5. Reconcile records

Run deterministic rules against the configured reference data; produce a result for each rule.

6. Create issue cases

Create reviewable issues with a rule code, description, severity, affected row, and current status.

7. Review and clarify

Reviewer inspects the issue and can request correction or additional evidence.

8. Correct and resubmit

Institution user corrects permitted fields; system preserves the change history.

9. Resolve the case

Reviewer records a decision and mandatory reason; system timestamps and audits the action.

10. View reports

Dashboard and downloadable report show totals that match stored submission and case data.

6. Main Modules and Screens

Module	Purpose
Dashboard	Submission totals, flagged records, open cases, recent activity, and status breakdowns.
Student Data Submission	Download template, upload CSV/XLSX, preview rows, validate, and display row-level errors.
Validation & Reconciliation Engine	Run transparent rule checks and explain why each record or count was flagged.
Institution & Programme Setup	Manage demo institutions, programmes, academic years, and configured intake references.
Review Workspace	Filter cases, inspect issue details, request correction, and record resolution with reason.
Evidence & Verification Status	Show evidence source and verification level; do not conflate a mock result with official verification.
Reports & Audit Trail	Export counts, issue summaries, decisions, and a traceable event history.
Security & Integration Settings	Role permissions, institution isolation, audit configuration, and clearly labelled integration status.

7. Functional Requirements

ID	Requirement
FR-01	Accept a documented CSV/XLSX student-record template.
FR-02	Validate required fields, types, allowed values, and formatting using transparent rules.
FR-03	Detect missing identifiers, malformed identifiers, duplicate rows, and duplicate identifiers according to approved rules.
FR-04	Compare counts only against configured authoritative reference data and applicable intake rules.
FR-05	Generate issue records with rule code, description, affected row/record, severity, and status.
FR-06	Support review, clarification requests, corrections, and resubmission.
FR-07	Require reviewer reasons for resolution or rejection.
FR-08	Record audit events for upload, validation, correction, review, export, and relevant administrative actions.
FR-09	Provide role-appropriate dashboards, search, filters, and reports.
FR-10	Keep demo/mock integrations visibly separate from verified official integrations.
FR-11	Enforce institution-level data separation and least-privilege access.
FR-12	Use synthetic data for the demonstration unless a lawful, approved data source is explicitly provided.

8. Non-Functional Requirements

- **Security:** secure authentication approach for the selected prototype environment, server-side authorization, protected secrets, and safe file handling.
- **Privacy:** collect only necessary fields; mask identifiers in the UI where appropriate; avoid real personal data in screenshots and demos.
- **Auditability:** important changes must be attributable to an actor and timestamp; avoid silently overwriting history.
- **Reliability:** repeatable validation results for the same input and rule configuration.
- **Usability:** clear error messages and reviewer actions understandable to non-technical staff.

- **Maintainability:** separate UI, API, data model, validation rules, and integration adapters.
- **Performance:** define measurable upload and response-time targets after expected file sizes and user loads are confirmed.
- **Accessibility:** readable contrast, keyboard-friendly controls, meaningful labels, and responsive layouts.

9. Roles and Access (Provisional)

Role	Proposed permissions
Institution user	Upload and view their institution's submissions; correct permitted data; respond to clarification requests.
Authorized reviewer	Review assigned/authorized cases, request clarification, record decisions, and view permitted reports.
Platform administrator	Configure users, demo institutions, reference data, and system settings; access must be restricted and audited.
External verifier	Not included by default. Add only if the problem owner confirms the role, legal basis, fields, and access rules.

These are proposed prototype roles, not a claim that they are official AICTE job titles or confirmed production permissions.

10. Data Model: Candidate Entities

The following is a starting point for schema design, not an approved government data dictionary. Every student field and its mandatory/optional status must be confirmed before production use.

Entity	Purpose / example attributes
institutions	Institution identifier, display name, institution type, status, and approved reference identifiers.
programmes	Programme/course identifier, institution relationship, academic year, and relevant category.
users / roles	Account identity, role, institution scope, status, and permission assignments.
students	Minimum approved student attributes and a protected APAAR identifier representation, if authorized and required.
enrolments	Student-to-institution/programme relationship, academic year, status, and relevant dates.
intake_reference	Approved intake figures, source, effective academic year, category rules, and provenance.
submissions	Uploader, institution, file metadata, submission time, row counts, and processing status.
validation_issues	Rule code, affected row/record, explanation, severity, and status.
verification_cases	Case owner, review status, decision, reason, and timestamps.
case_events / evidence_records	Correction/review history, evidence references, source, and access restrictions.
audit_logs	Actor, action, timestamp, affected entity, and relevant change metadata.

11. Proposed Technical Architecture

Suggested prototype stack (subject to team capability and deployment constraints):

- **Frontend:** React + TypeScript + Tailwind CSS for dashboard, upload, review, and reporting screens.
- **Backend API:** Python FastAPI for authentication/authorization hooks, submissions, validation, cases, reports, and audit events.
- **Database:** PostgreSQL for relational data and traceable workflow state.
- **File processing:** Pandas and openpyxl for CSV/XLSX parsing with file-size, extension, and content validation.

- **Testing:** pytest for validation/business rules and API tests; frontend tests where practical.
- **Integration boundary:** a mock adapter for demo responses; a real adapter only after written authorization and official technical documentation.

Use deterministic, explainable validation for the MVP. An LLM or machine-learning model is not necessary to detect missing fields, duplicates, or configured count discrepancies.

12. Proposed Status Model

Object	Proposed states
Submission	Uploaded → Validating → Processed with issues / Processed cleanly → Corrected or superseded
Issue / case	Open → Under review → Clarification requested → Resubmitted → Resolved / Rejected with reason
Verification evidence	Not checked → Format checked → Institution-attested → Official source checked (only when actually performed)

Do not display 'officially verified' for a local format check or a simulated API response.

13. Prototype Test Plan and Acceptance Criteria

Test	Pass condition
Missing field	Upload with a required field blank; the correct row and field are reported.
Malformed APAAR ID	Flag according to the confirmed format specification; do not call it identity verification.
Duplicate within file	Detect duplicate rows/identifiers according to the configured rule and show the affected rows.
Possible cross-institution match	Flag for authorized review; do not automatically label fraud.
Intake mismatch	Compare against the selected reference year/category; display the difference and rule used.
Correction history	After a correction, show who changed what and when, subject to approved retention rules.
Reviewer decision	Prevent final resolution without a permitted reviewer and required reason.
Institution isolation	A user from Institution A cannot view or modify Institution B's records through UI or direct API requests.
Report consistency	Dashboard, export, and stored case counts agree for the same filters and timestamp.
Mock integration	Every simulated external response is visibly labelled as demo/mock; no real API is implied.

14. Predicted Outcomes and How to Measure Them

The prototype is expected to demonstrate whether a transparent workflow can make common data-quality issues easier to detect, review, correct, and report. These are hypotheses to measure—not guaranteed results or claims of production impact.

Outcome	Suggested measurement
Issue detection	Percentage of seeded test issues correctly detected by the rules.
False flags	Number of clean records incorrectly flagged in the controlled test set.
Traceability	Percentage of review/correction actions with actor, timestamp, and reason where required.
Isolation	All negative cross-institution access tests are denied.
Reporting accuracy	Dashboard/export totals match the underlying test data.

Outcome	Suggested measurement
Workflow efficiency	Compare task completion time with and without the prototype using the same test cases.

Do not invent percentage improvements. Measure them during prototype testing and report the test-set size, rule configuration, and limitations.

15. Implementation Plan

Phase 0 — Confirm scope

Review the official problem statement and available manuals; list confirmed facts, assumptions, and unanswered questions.

Phase 1 — Data dictionary & rules

Define every field, data type, required/optional status, source, validation rule, and allowed value. Keep unconfirmed fields marked provisional.

Phase 2 — Schema & API contract

Design entities, relationships, permissions, endpoints, error format, and audit events.

Phase 3 — Validation engine

Implement small deterministic rules with unit tests and synthetic fixtures before building the full UI.

Phase 4 — Core workflow

Implement submission, issue listing, review, correction, resolution, and audit history.

Phase 5 — Dashboard & reports

Build metrics from stored data and verify export consistency.

Phase 6 — Security & end-to-end tests

Test role permissions, institution isolation, upload safety, correction history, and all acceptance criteria.

Phase 7 — Demo preparation

Use synthetic data, scripted scenarios, labelled mock integration, and a clear explanation of limitations.

16. Questions That Must Be Confirmed Before Production

- What is the official student-level schema for each institution type, and which fields are mandatory?
- What is the authoritative source for approved intake, and how do categories, supernumerary seats, NRI/OCI/FN, and other exceptions affect counts?
- Is there an officially supported APAAR ID or current-enrollment API for this exact use case? Who authorizes access and supplies documentation/credentials?
- How should transfers, withdrawals, repeat students, concurrent programmes, and changes of institution be treated?
- Who may see which student fields, and what permissions apply to external verifiers or other regulators?
- What does 'real-time' mean for this use case, and what evidence qualifies a record as 'verified'?
- What are the hosting, security, retention, deletion, consent/legal-basis, audit, incident-response, and reporting requirements?
- What performance targets, file sizes, expected user counts, and official report formats must be supported?

17. Privacy, Security, and Integration Boundaries

- Do not scrape APAAR or other official portals, bypass access controls, or assume an API exists. Obtain explicit authorization and follow applicable terms.
- Use synthetic records in development, screenshots, presentations, and demonstrations. Do not use real student personal data without an approved lawful process.
- Apply least privilege, institution-level isolation, server-side authorization, secure secrets handling, safe file processing, and audit logging.
- Minimize exposure of APAAR identifiers in the UI and logs; define encryption, masking, retention, and deletion controls with the responsible authority.
- Obtain qualified review of applicable privacy and education regulations, including commencement and applicability of legal provisions, before production deployment.
- Treat a possible duplicate as a review signal, not a fraud verdict; allow legitimate transfers and status changes to be explained.

18. Source Register and Research Starting Points

These links are starting points for validation. A third-party listing should not be treated as the authoritative source of the problem statement. Confirm the exact SIH statement and all production requirements with the official problem owner.

- **Official SIH 2026 portal:** https://sih.gov.in/sih2026PS
- **SIH26263 third-party listing (cross-check against official source):** https://zaidsayyed.in/tools/sih-problem-statements/sih26263
- **AICTE Student Enrollment submission manual:** https://www.aicte.gov.in/sites/default/files/User_Manual_for_Submitting_Student_Enrollment_Placement_New_Initiative_General_Information.pdf
- **AICTE Approval Process Handbook 2024–2027:** https://www.aicte.gov.in/sites/default/files/APH%20Final.pdf
- **APAAR official portal:** https://apaar.education.gov.in/
- **APAAR Terms of Use:** https://apaar.education.gov.in/termsofuse
- **APAAR Privacy Policy:** https://apaar.education.gov.in/privacy-policy
- **APAAR FAQs:** https://apaar.education.gov.in/faqs
- **NAD/DigiLocker Verifier:** https://nad.digilocker.gov.in/Verifier
- **NAD FAQs:** https://nad.digilocker.gov.in/faq
- **AISHE:** https://aishe.gov.in/
- **API Setu:** https://apisetu.gov.in/
- **SAMARTH:** https://samarth.edu.in/
- **UDISE+:** https://udiseplus.gov.in/

19. Instructions for Teammates and AI Coding Agents

Treat this document as the shared project brief. Before implementing a feature, identify which requirement it satisfies, what data it reads/writes, which role can use it, how it is tested, and what evidence demonstrates completion.

- Do not silently invent official fields, business rules, legal requirements, APIs, government approval, or integration behavior.
- Keep confirmed source facts, project decisions, and assumptions clearly separated.
- Implement transparent deterministic rules first; every rule must have a name, explanation, input, expected result, and test.
- Use synthetic seed data and label demo-only data/features clearly.

- Do not mark a record officially verified unless an authorized, real source check was actually performed.
- Do not automatically accuse an institution or student of fraud based on a duplicate or count mismatch.
- Keep UI, API, schema, validation rules, and integration adapters separated and documented.
- For each completed task, provide changed files, how to run it, tests executed, known limitations, and any unresolved requirement.

20. Final Project Definition

APAAR ReconX is a proposed, security-conscious reconciliation and review platform that helps authorized users validate submitted student records, identify enrollment discrepancies against properly configured references, resolve issues through a traceable human workflow, and produce consistent reports. The prototype demonstrates this workflow using synthetic data and mock integrations; production integration and official policy rules remain subject to confirmation and authorization.

Document note: This is a project understanding and implementation guide, not an official AICTE/APAAR specification, legal opinion, or authorization to access government systems.